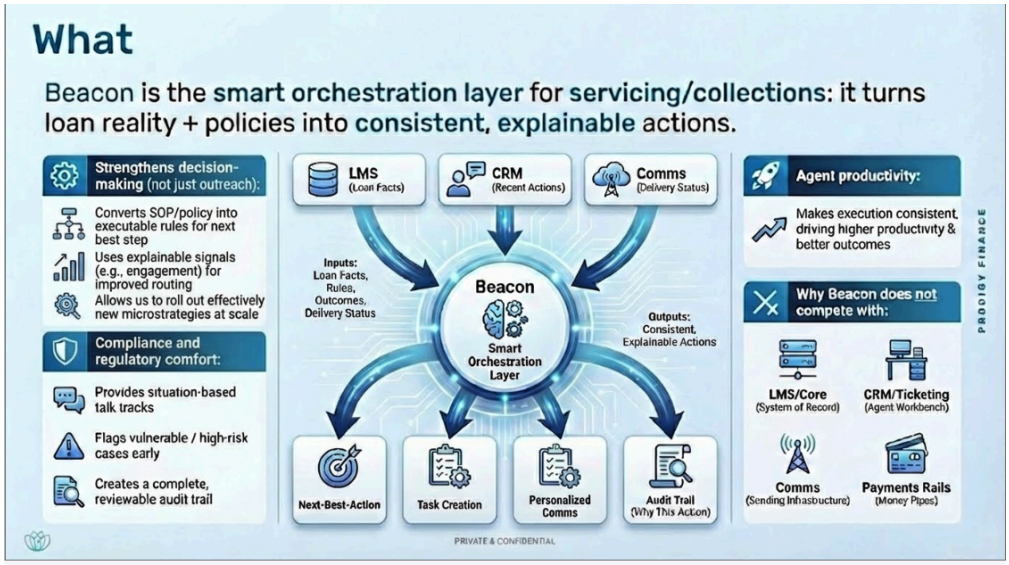
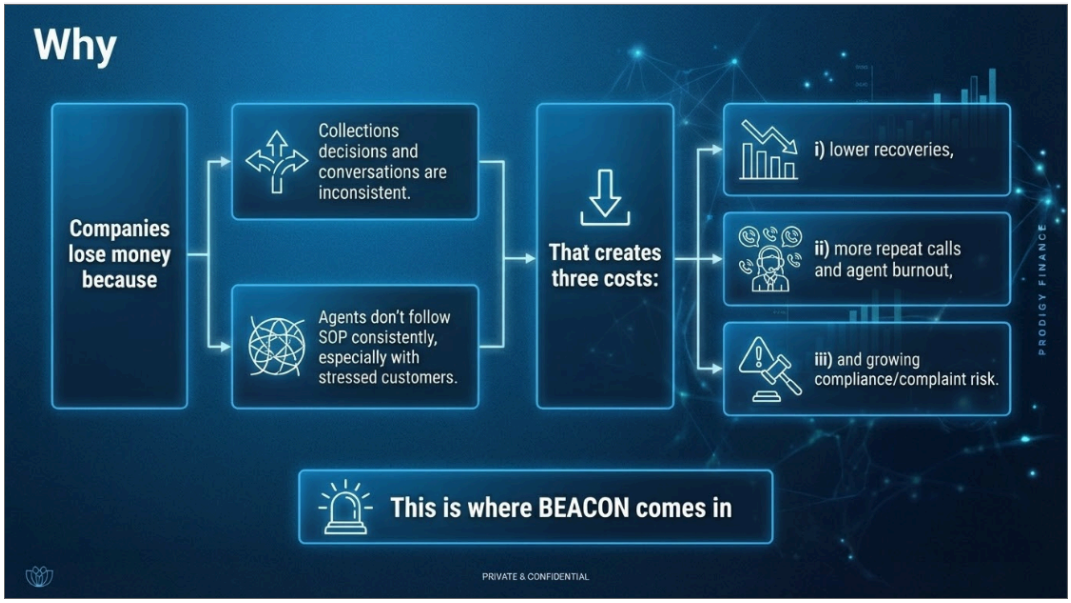
Project Title: Project Beacon

Core Objective:

Build a B2B web application for lenders to upload CSV/JSON customer data (loans, payments, chat logs). The app must use an AI agent to analyze this data against a specific SOP (Standard Operating Procedure) to recommend a decision (e.g., Approve, Deny, Flag for Review).

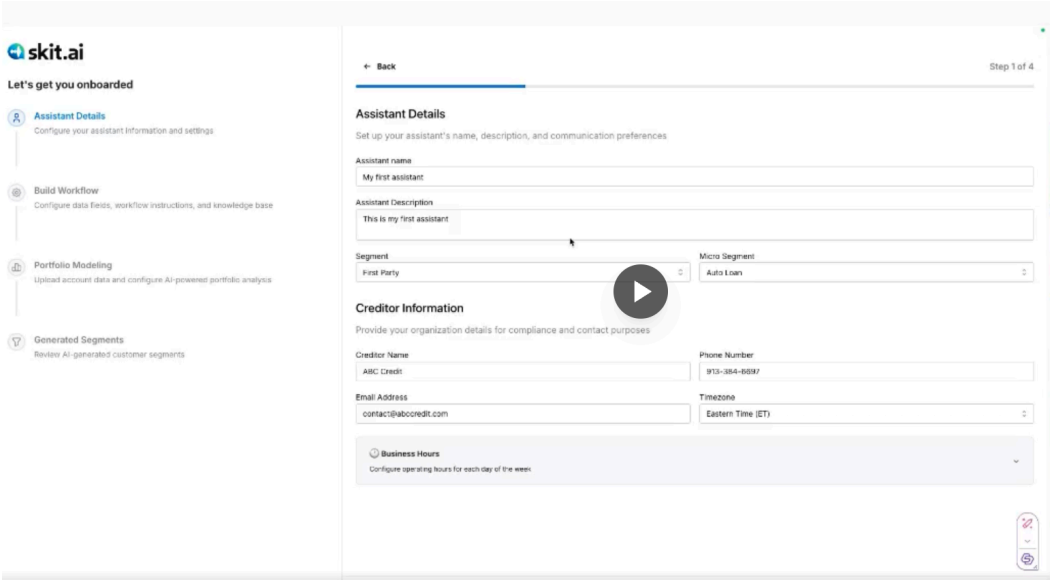


Why are we building this

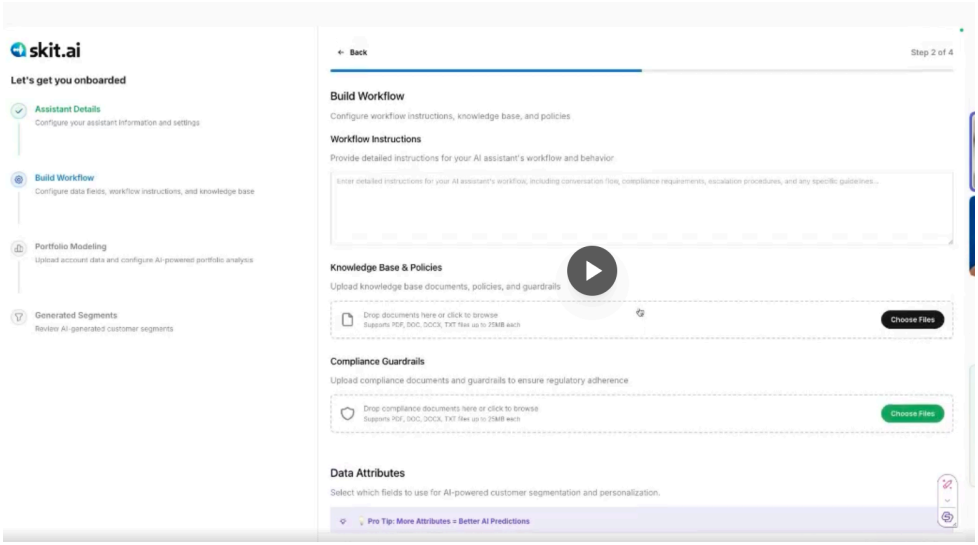


Key Features:

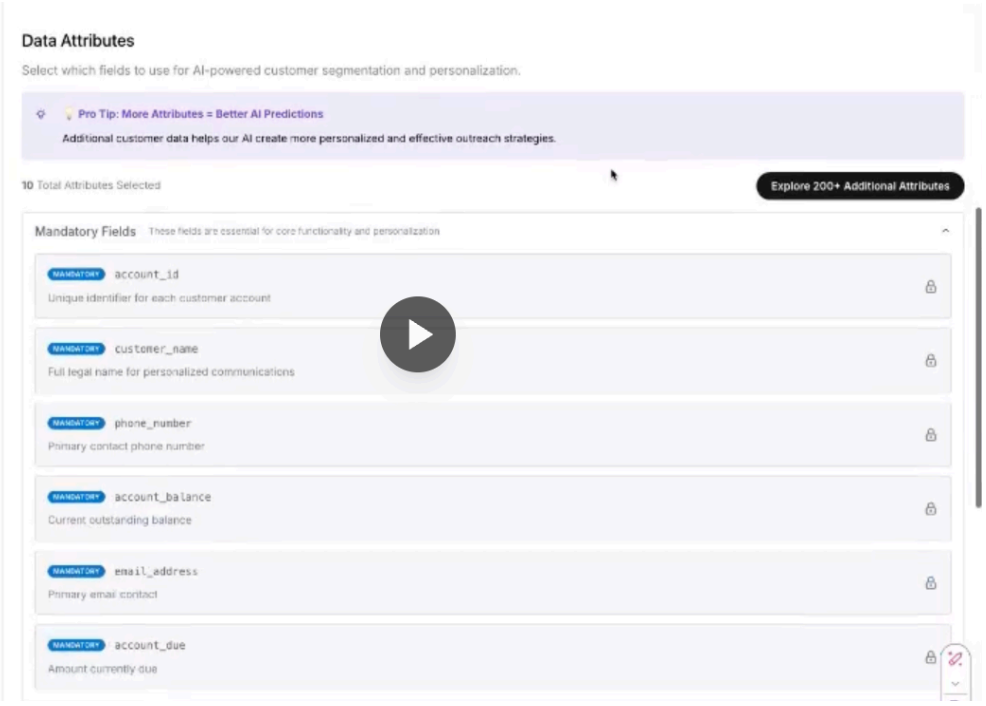
- **Website:** landing page that communicate our value props correctly, honors Prodigy Finance branding (https://prodigyfinance.com/) and a reference website that could be helpful: https://www.flexys.com/ . Additionally we need a way for our customer to login and Register through email , password and confirm password
- **Client Configuration:** This is basically client setting up their account , providing email id contact details and name of the company

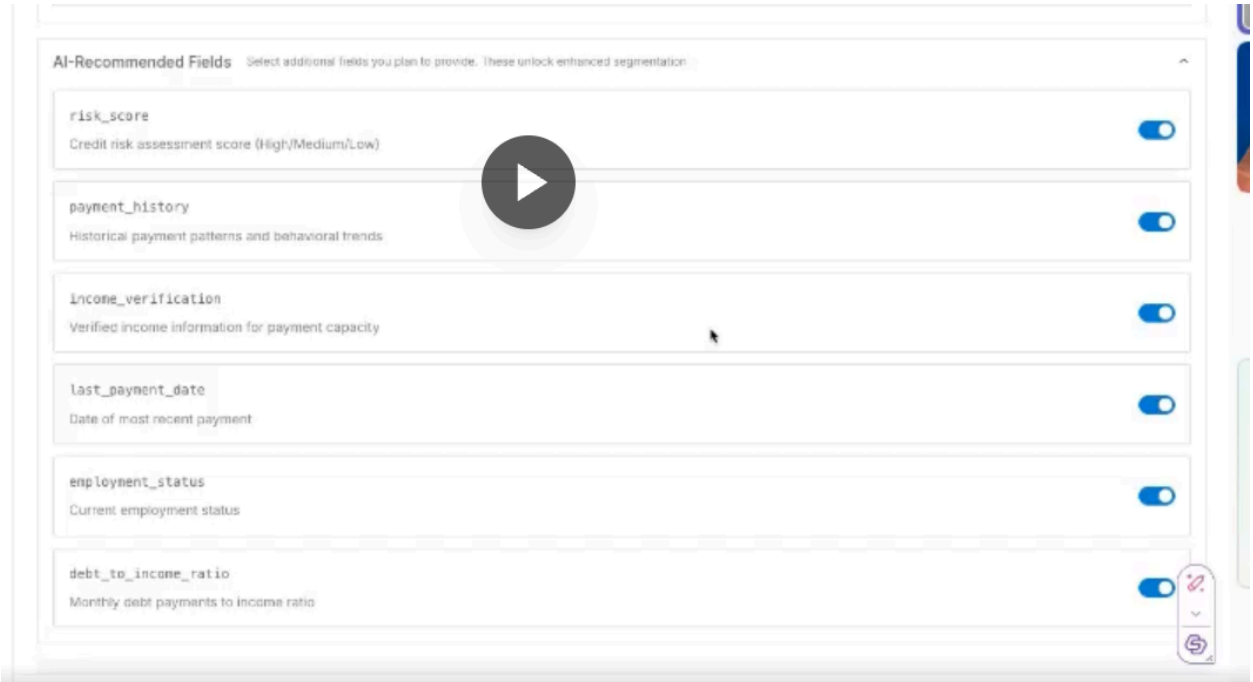


Action Rulebook: Policy/SOP: current early-delinquency playbook (rules, do's/don'ts) This is where rules will come in for example: if the customer is paying 0.3<CMD and also has mentioned about employment loss- offer forbearance. It may also have some guideline how to interpret data etc. Create space to input free text or upload PDF or JPG which will be fed along with customer data to AI to determine the action on the customer. *Store the SOP as a text-based system prompt for the LLM."* Handling JPGs/PDFs requires using Gemini OCR



Then comes configuring what data will they provide





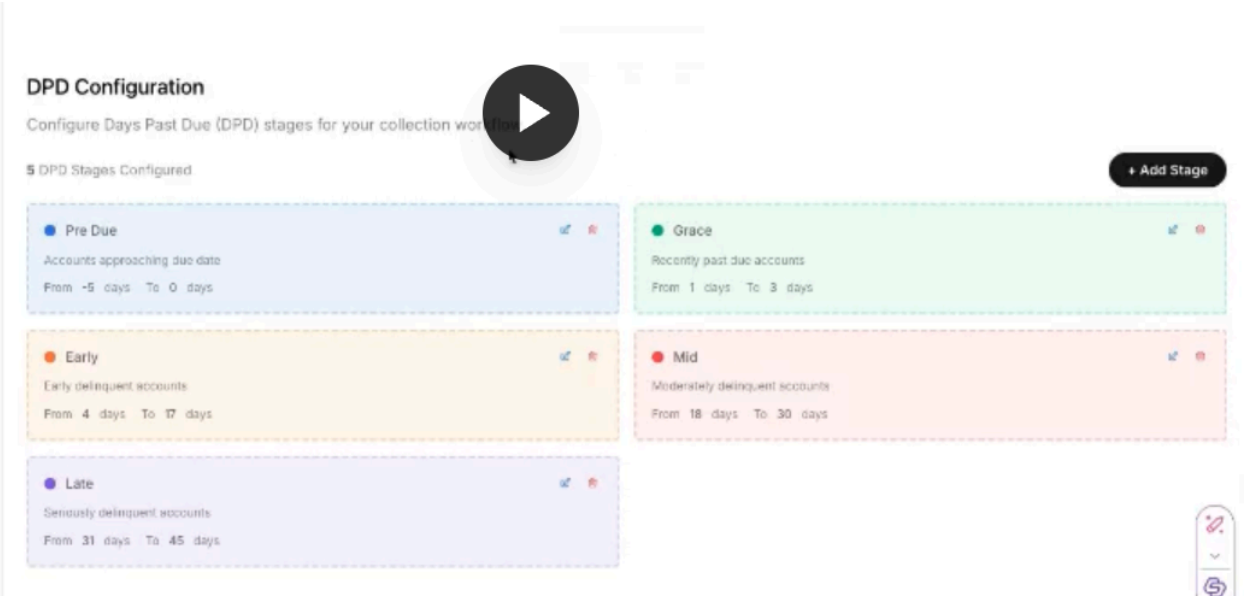
Minimum Data Pack (mandatory for pilot)

- Loan/account: customer/account ID, DPD (B1–B7), outstanding, current due amount/date
- Payments: Customer ID, payment amount, status of payment, date of Payment (history till date or last 6 months)

Optional (improves accuracy, not required to start)

- Customer conversation history / contactability signals
- Income/employment attributes
- Bureau attributes
- Arrangements/PTPs/payment plans + broken promises
- Comms/compliance policy + knowledge base

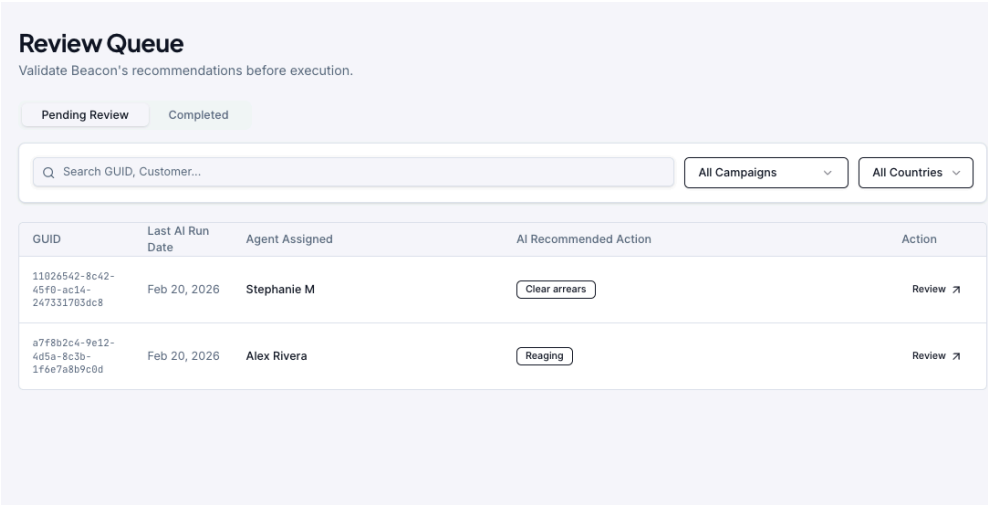
Then comes configuration DPD values



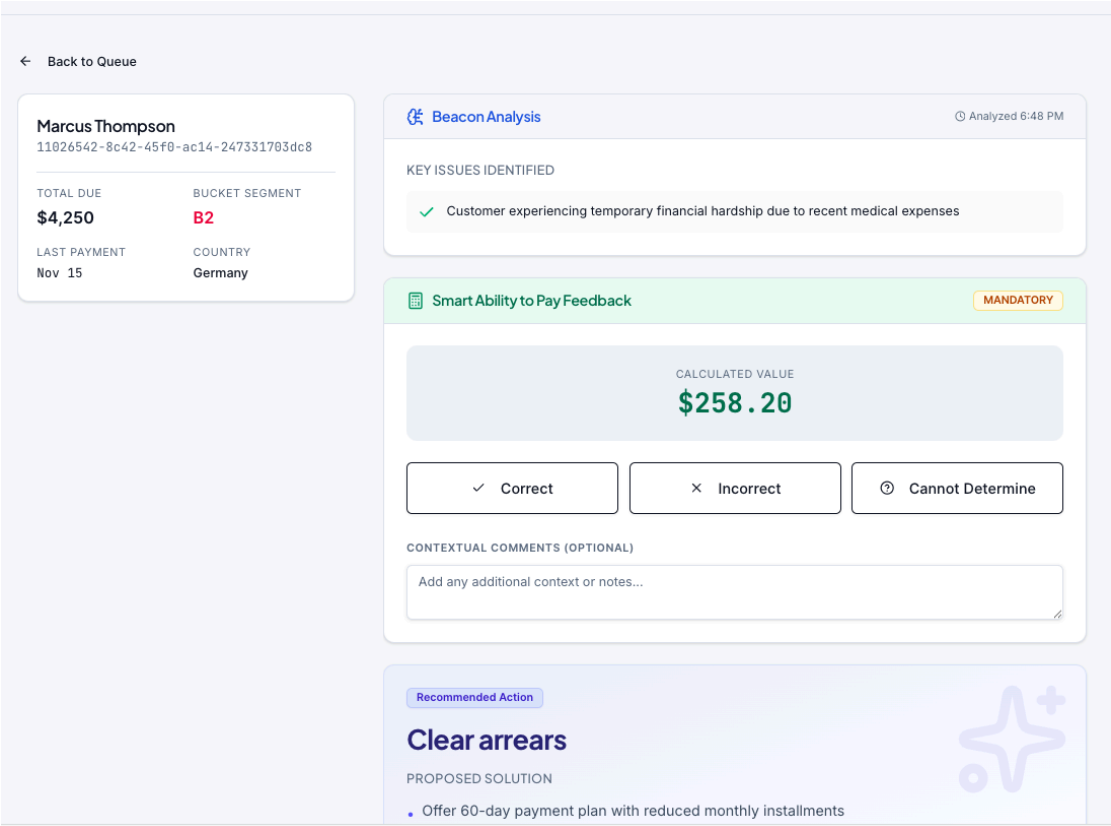
- **Secure File Upload: Once client configuration is done then we have a new tab called Data ->where they can** upload the data via and showcases the SOP or knowledge docs uploaded

- **AI Decision Engine:** Integration with LLMs to Gemini 3.0 that will be read data one customer at a time all customer data for that data along with SOP then provides an Output. There needs to be a place to configure prompt- which basically tells what all data to be read by AI and how the output should look like that needs to be passed onto AI and this is specific to that client

- **Review Queue:** This is where customers will show up if there is an action that needs to be taken A view showing "Pending Decisions" and "Decision History."



How agents will view the decision made (output from AI) and customer data in single view.



Agent can then choose if they agree or not. If they don’t take the right action then can tell us why they don’t agree with us

Output from AI prompt

customer_guid: UUID from input.

* **Combined_CMD**: A single numerical value. **(See Rulebook Section 1.1)**

* **problem_description**: ≤5 lines.

* **problem_confidence_score**: Integer 1-10. **(See Rulebook Section 4.5)**

* **problem_evidence**: ≤5 lines. Cite specific data sources (e.g., specific dates, transaction amounts, or interaction details) to support your claim.

* **proposed_solution**: ≤5 lines. **(See Rulebook Section 3.1)**

* **solution_confidence_score**: Integer 1-10. **(See Rulebook Section 4.5)**

* **solution_evidence**: ≤5 lines.

* **internal_action**: ≤5 lines.

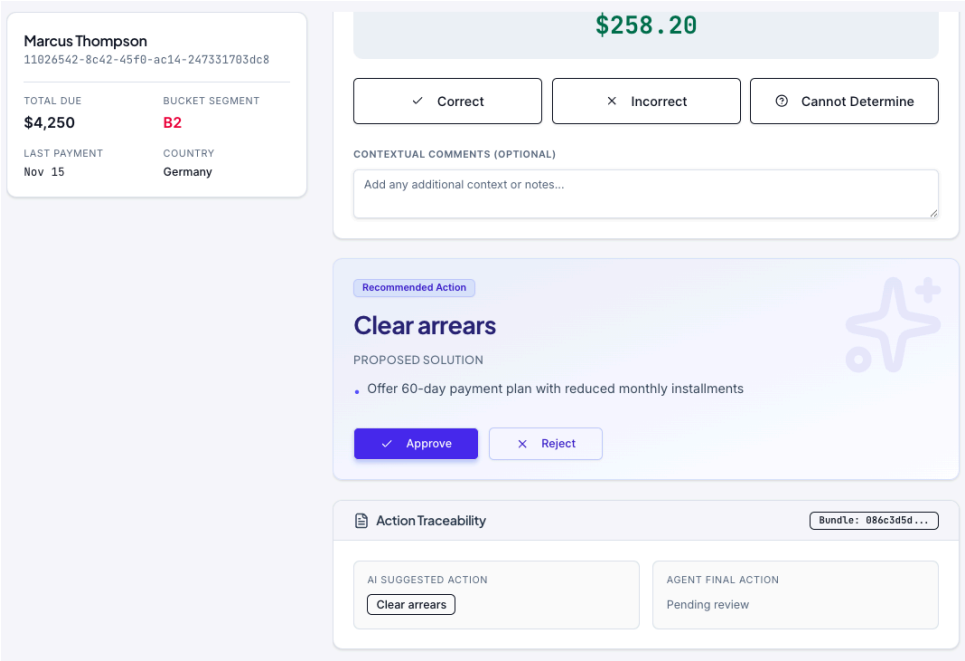
* **Internal Action Normalization (MANDATORY):**

* **ability_to_pay**: A single numerical value.

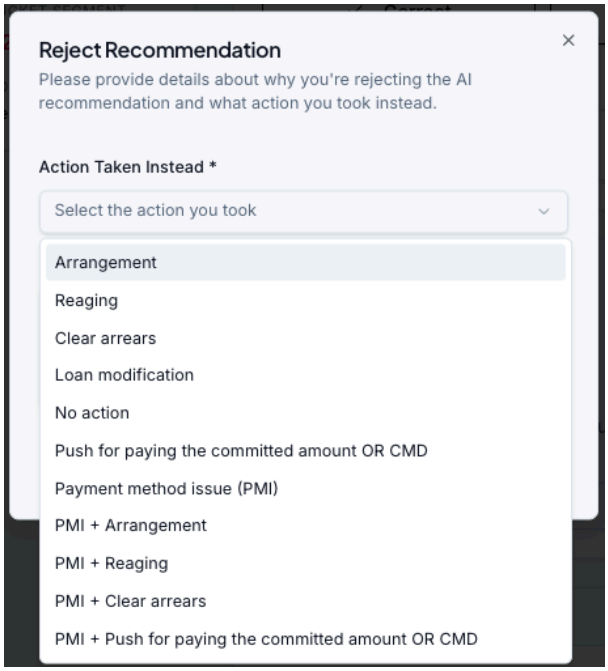
* **reason_for_ability_to_pay**: ≤5 lines justification, citing the rule used or explicit customer declarations. **(See Rulebook Section 1.2)**

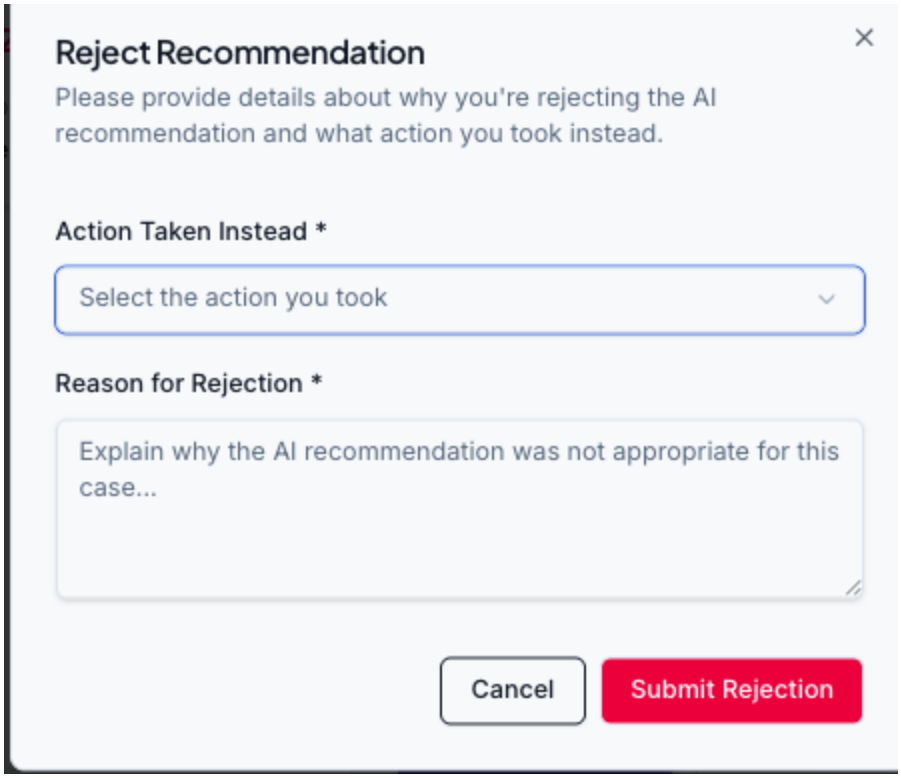
* **no_of_latest_payments_failed**: An integer. The count of the most recent consecutive failed payments from real_time_payments.csv. **(See Rulebook Section 1.6)**

* **proposed_email_to_customer**: Full email text (Subject: and Body:) OR the text `NO_ACTION`.



We would like to show proposed email also from AI and similar have accept and reject and provide the reason in case they reject

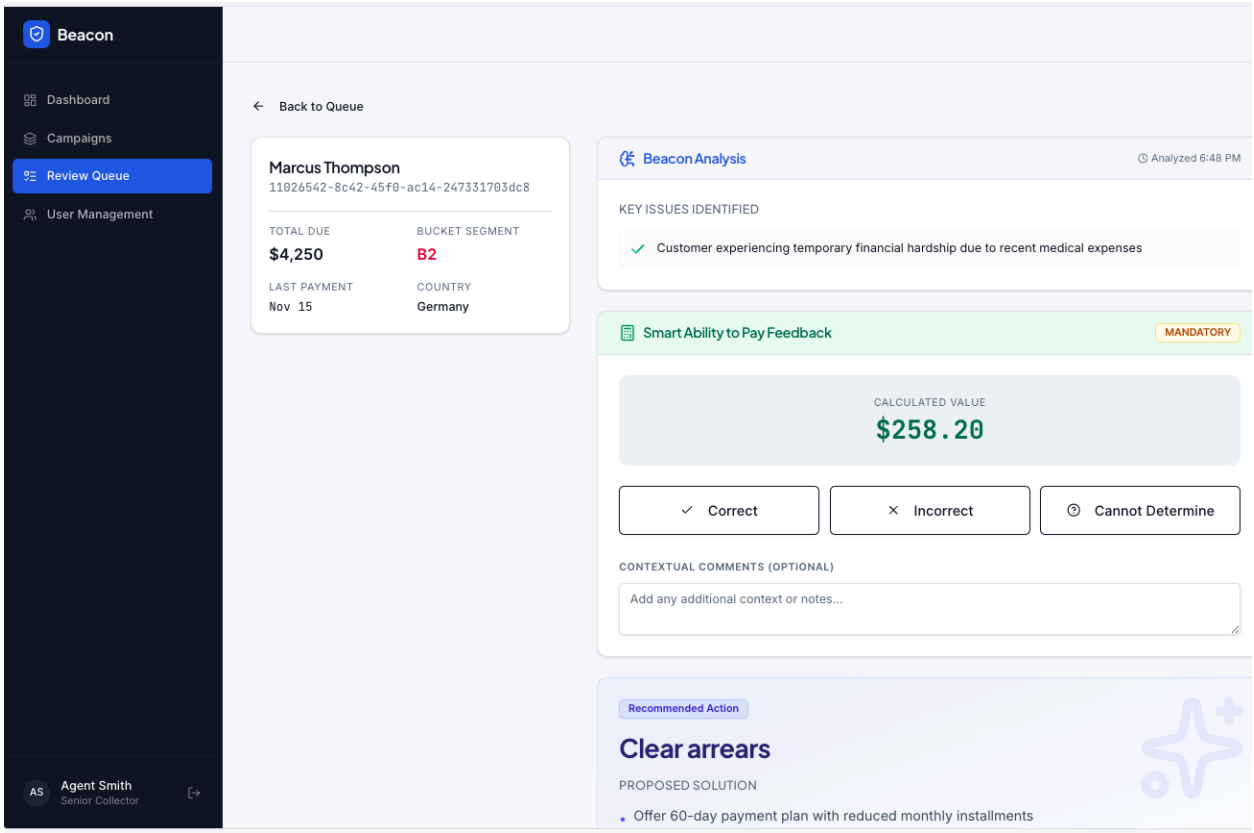




User Roles and Permissions

- To keep it simple, Whoever registers will be manager role and will configure the rules, upload the customer data and also sees the review queue

Miscellaneous



Frontend: React with Tailwind CSS.

Backend: Python (FastAPI) or Node.js.

Database: PostgreSQL (via Replit's managed DB).

Security: Ensure data is handled as sensitive financial information.

We have specified **Gemini 3.0**. Please use the **Google Cloud Vertex AI SDK** or the **Google AI SDK** to ensure it integrates correctly with the model you want.